

BDCC Midterm Reviewer

Sep 28, 2026 · @Robin

Exam overview

The midterm is split 60% multiple-choice (concepts) / 40% hands-on in Jupyter, and the prof said it's based directly on the lecture exercises. That means the hands-on half will look like the FIM, Neighborhood-based CF, and Latent Factor CF exercise notebooks — write a function, get a result, maybe interpret it for a business case (the same shape as Assignment 1 on the Sushi dataset).

Scope (matches the four lecture decks, up to the point the course outline marks "Midterm"):

1. Intro to Big Data & Cloud Computing
2. Frequent Itemset Mining (FIM) & Association Rule Mining (ARM)
3. Neighborhood-based Collaborative Filtering (NB-CF)
4. Latent Factor-based Collaborative Filtering (LF-CF)

How to use this reviewer

- Read a topic section below, then go to its "worked by hand" example and re-derive every number yourself with pen and paper before checking it against what's shown — that's the step-by-step manual pass you asked for.
- Use the MCQ self-check section to test recall of definitions, formulas, and algorithm trade-offs.
- Use the companion review notebook (sent as a file, and saved to your BDCC 3301 folder) for the 40% hands-on half — it has fresh toy data (not the same numbers as your solved exercises) so you're practicing recall, not remembering answers.

Big Data & Cloud Computing fundamentals

The V's of Big Data. Core 3 V's: Volume, Velocity, Variety. Extended sets add Veracity (data quality/trustworthiness), Value, Variability, Visualization, Validity, Vulnerability, Volatility — different papers stop at 4/5/7/10 V's. MCQs may ask you to match a V to a definition or count.

What makes Big Data special (3 reasons):

1. It needs vast computing resources (storage, memory, processor) — handled via horizontal scaling (more machines) rather than only vertical scaling (bigger machines).
2. It needs parallel and distributed computing — this is where MapReduce and Hadoop come in.
3. It enables training higher-capacity ML models — more data lets you move up the performance curve for high-capacity algorithms (deep nets) versus low-capacity ones, which plateau early.

MapReduce. A programming model for processing large datasets in parallel across a cluster: a **Map** step transforms input records into key-value pairs independently (parallelizable), then a shuffle/sort groups by key, then a **Reduce** step aggregates values per key. Fault-tolerant and scales horizontally.

Hadoop & data locality. Hadoop implements MapReduce on top of HDFS (Hadoop Distributed File System), which stores data in blocks replicated across nodes. **Data locality** = moving computation to where the data already lives instead of moving data across the network, ranked cheapest to most expensive: in memory → local SSD → local HDD → over the network on the same rack → over the LAN → over the internet. This is why Hadoop schedules tasks on (or near) the node holding the data.

Apache Spark. A faster alternative/successor to plain MapReduce — keeps data in memory (RDDs/DataFrames) between operations instead of writing intermediate results to disk each step, which is why it's much faster for iterative algorithms (like the CF methods below).

Cloud Computing. A shared pool of computing resources (compute, storage, networking) provisioned with minimal management effort. Benefits: pay-as-you-go pricing, shifting spend from CapEx to OpEx, and elastic scalability. Know the service-model layers referenced in the deck: **IaaS** (e.g., EC2, EBS — you manage the OS/runtime up), **PaaS** (e.g., EMR, S3-backed managed services), and how these differ from on-prem where you own everything including hardware.

Frequent Itemset Mining & Association Rule Mining

Core definitions

- Database T : a set of N transactions, each a subset of a universe U of d items.
- **Itemset X** : any subset of U ; a **k -itemset** has $|X| = k$.
- **Absolute support**: $\text{sup}(X) = \text{number of transactions containing } X \text{ as a subset}$.

- **Relative support:** $\text{relSup}(X) = \text{sup}(X) / N$ (fraction of transactions). Most libraries (incl. pyFIM) define support as relative by default; a positive `supp` argument in `fim` is a percentage, negative is absolute count.
- **Frequent itemset:** one whose support meets a minimum threshold `minsup`.
- **Maximal frequent itemset:** a frequent itemset with no frequent superset (not a subset of any other frequent itemset returned).

Three algorithms for the same problem (all correct, differ only in efficiency):

Algorithm	Idea	Worst-case complexity	Practical speed
Brute force	Enumerate every possible itemset, count support of each	$O(N \times 2^d)$	Slowest, unusable once d grows
Apriori	Level-wise (k -itemsets to $(k+1)$ -itemsets) using the downward closure property: every subset of a frequent itemset must also be frequent, so only extend itemsets built from frequent k -itemsets	Same worst case as brute force	Faster in practice, prunes candidates early, but still generates candidates that may not exist in the data
FP-Growth	Builds a prefix tree (FP-tree), mines via projected databases: recursively restrict the database to transactions containing an item, remove that item, repeat	Same worst case	Fastest in practice, never generates a candidate that isn't actually in the data (pattern-growth, not candidate-generate-and-test)

Key MCQ trap: increasing the number of items (d) hurts runtime exponentially; increasing the number of transactions (N) only hurts linearly. So scaling items is far more expensive than scaling transactions.

Association Rule Mining (usually the next step after FIM). For a rule $A \rightarrow B$:

$$\text{conf}(A \rightarrow B) = \text{sup}(A \cup B) / \text{sup}(A)$$

Confidence = conditional probability of B given A.

$$\text{lift}(A \rightarrow B) = \text{conf}(A \rightarrow B) / \text{relSup}(B)$$

Lift = how much more likely B is when A is present vs. its baseline rate. Lift = 1 means independence; greater than 1 means positive association; less than 1 means negative association.

A is the **antecedent**, B the **consequent**. Rules are filtered by `minsup` and `minconf` thresholds, then typically ranked by lift.

pyFIM is the go-to library since pandas/scikit-learn don't have FIM built in. Know: `fim.apriori`, `fim.eclat`, `fim.fpgrowth`; `supp` (percent if positive, absolute if negative), `zmin` (minimum itemset length), `target='r'` + `report='l'` to get rules with lift directly.

Worked by hand: Apriori

Toy database, `minsup = 2` (absolute count), 4 transactions:

```
id,items
T1,"bread,jam"
T2,"bread,jam,milk"
T3,"bread,milk"
T4,"jam,milk"
```

Cover this cell before you look, then trace it yourself:

Layer k=1 (count each single item across all 4 transactions):

- bread: in T1, T2, T3 -> support 3
- jam: in T1, T2, T4 -> support 3
- milk: in T2, T3, T4 -> support 3

All three pass `minsup=2`, so all three 1-itemsets survive to generate candidates for `k=2`.

Layer k=2 (combine surviving 1-itemsets pairwise): candidates {bread,jam}, {bread,milk}, {jam,milk}

- {bread,jam}: in T1, T2 -> support 2 (pass)
- {bread,milk}: in T2, T3 -> support 2 (pass)
- {jam,milk}: in T2, T4 -> support 2 (pass)

All three pass again.

Layer k=3: only candidate is {bread,jam,milk} (built by extending a frequent 2-itemset with an item that keeps all its subsets frequent).

- {bread,jam,milk}: needs all three items in one transaction. Only T2 has all three -> support 1 (fails, since $1 < 2$).

No candidates survive to $k=4$, so Apriori stops. **Frequent itemsets:** three 1-itemsets (support 3 each) and three 2-itemsets (support 2 each) = 6 total. The **maximal** frequent itemsets are just the three 2-itemsets, since none of them is a subset of another frequent itemset.

This is exactly what the downward closure property buys you: because {bread,jam,milk} failed, Apriori never had to check any 4-itemset (there isn't one here, but in a larger universe this pruning is what saves the runtime).

Worked by hand: FP-Growth (same toy database)

Same database and minsup=2: bread(sup 3), jam(sup 3), milk(sup 3) — all frequent, so the FP-tree keeps all items.

Step 1: order items by decreasing support (ties broken alphabetically here since all three tie at 3): bread, jam, milk.

Step 2: recursive pattern growth, starting with an empty prefix and the full database D:

1. Count each item in D: bread=3, jam=3, milk=3. All pass minsup=2.
2. Take the first frequent item, bread. Emit itemset {bread} with support 3.
3. Build bread's **projected database**: keep only transactions containing bread, drop bread itself, keep only items that sort after bread (jam, milk): from T1{jam}, T2{jam,milk}, T3{milk} → projected D_bread = [{jam}, {jam,milk}, {milk}].
4. Recurse on D_bread with prefix (bread,): count jam=2, milk=2, both pass → emit {bread,jam} sup 2 and {bread,milk} sup 2. Project again on jam inside D_bread: only {jam,milk} contains jam → {milk} with support 1 → fails minsup, so {bread,jam,milk} is NOT emitted (matches the Apriori trace above, where this itemset also failed at $k=3$).
5. Back at the top level, move to jam: emit {jam} sup 3, project D_jam (transactions with jam, keeping only items after jam — just milk): from T1{}, T2{milk}, T4{milk} → D_jam = [{}, {milk}, {milk}] → milk has support 2 → emit {jam,milk} sup 2 (already found via the

bread branch too — FP-Growth's recursion naturally reaches the same itemset from different starting prefixes, but a correct implementation only emits each itemset once).

6. Finally milk on its own: emit {milk} sup 3 (no items sort after milk, so no further projection).

Result: same 6 frequent itemsets as Apriori/brute force — {bread}, {jam}, {milk} (sup 3 each), {bread,jam}, {bread,milk}, {jam,milk} (sup 2 each). The difference from Apriori is *how we got there*: FP-Growth never built or counted a 3-itemset candidate at all — the projected database for {bread,jam} only ever contained {milk} with support 1, so the search simply never reached {bread,jam,milk}, instead of generating it as a candidate and then discovering it fails (which is what Apriori does).

Neighborhood-based Collaborative Filtering

Utility matrix: rows = users, columns = items, cell = rating (mostly missing/sparse). Goal: predict the missing cells (equivalently, "complete the matrix") well enough to rank unseen items for each user.

Two families:

- **Neighborhood-based / memory-based** (this section): use raw ratings directly, find similar users or items.
- **Model-based / latent factor** (next section): learn a compressed representation instead.

Similarity measures

Raw cosine similarity between users u and v , computed only over items both have rated (I_{uv}):

$$\cos(u,v) = (\sum_i r_{ui} \cdot r_{vi}) / (\sqrt{\sum_i r_{ui}^2} \cdot \sqrt{\sum_i r_{vi}^2}), \text{ summed over } i \text{ in the items both } u \text{ and } v \text{ rated}$$

Problem: raw cosine is fooled by users who rate everything high (or low) — it reads as "similar taste" when it's really just a rating-scale difference. Fix: **mean-center first** (subtract each user's average rating from their ratings), then take cosine. This is the **adjusted cosine similarity**:

$$\text{AdjCos}(u,v) = (\sum_i (r_{ui} - \mu_u)(r_{vi} - \mu_v)) / (\sqrt{\sum_i (r_{ui} - \mu_u)^2} \cdot \sqrt{\sum_i (r_{vi} - \mu_v)^2})$$

where μ_u , μ_v are the mean ratings of u and v . Only overlapping (commonly rated) items are used — never impute missing values with a constant before computing similarity, that would bias the result.

User-based CF algorithm:

```
for every user u:
    find k most similar users (peer group), by adjusted cosine
    for every unrated item i of u:
        predicted rating =  $\mu_u$  + weighted average of (peer's mean-centered rating on i), weighted by similarity
```

$$\hat{r}_{ui} = \mu_u + \left[\sum_v \text{sim}(u,v) \cdot (r_{vi} - \mu_v) \right] / \left[\sum_v \text{sim}(u,v) \right], \text{ summed over } v \text{ in the } k \text{ most similar users to } u \text{ who rated } i$$

Item-based CF is the mirror image: find the k most similar *items* to i (by the same similarity formula, but comparing columns instead of rows), then predict a user's rating on i as a similarity-weighted average of that user's ratings on similar items:

$$\hat{r}_{ui} = \mu_u + \left[\sum_j \text{sim}(i,j) \cdot (r_{uj} - \mu_u) \right] / \left[\sum_j \text{sim}(i,j) \right], \text{ summed over } j \text{ in the } k \text{ most similar items to } i \text{ that } u \text{ has rated}$$

User-based vs. item-based, why they're not symmetric in practice:

- Once you've found similar users, you already have your recommendation (their high-rated items); with similar items you still need a separate step to combine them per user.
- Items often cluster naturally by genre/category, so item-item similarity tends to be more stable than user-user similarity (a user can like several genres at once, diluting user similarity).

Evaluating recommenders

- **Online evaluation:** real users interact with the live system (e.g., A/B test) — expensive, most realistic.
- **Offline evaluation:** reuses already-collected ratings — cheap, most common in research/coursework, but can be stale.
- **Rating accuracy** metrics: MSE, RMSE, MAE, MAPE (treat prediction as regression).

- **Ranking/utility** metrics: what matters in practice is whether the *top few* recommended items are good, with earlier ranks weighted more — that's what DCG and NDCG measure.

$$DCG = (1/m) \sum_u \sum_j (2^{r_{uj}} - 1) / \log_2(v_j + 1)$$

where L = number of recommended items, v_j = the rank (1st, 2nd, ...) of item j in the recommendation list, r_{uj} = the actual rating user u gave item j .

$$NDCG = DCG / IDC$$

IDC = the DCG you'd get from the *ideal* ranking (items sorted by true rating, descending). NDCG normalizes DCG to $[0,1]$ so it's comparable across users with different rating scales/counts — 1.0 means your ranking matched the ideal ranking exactly.

Worked by hand: user-based CF

Toy utility matrix (blank = unrated):

User	Movie1	Movie2	Movie3
A	4	2	-
B	5	3	4
C	1	-	5

Goal: predict A's rating on Movie3, using B and C as neighbors ($k=2$, adjusted cosine).

Step 1: means. $\mu_A = (4+2)/2 = 3$. $\mu_B = (5+3+4)/3 = 4$. $\mu_C = (1+5)/2 = 3$.

Step 2: mean-center. A: $[1, -1, -]$. B: $[1, -1, 0]$. C: $[-2, -, 2]$.

Step 3: adjusted cosine similarity, A vs. B (common items: Movie1, Movie2):

- numerator = $(1)(1) + (-1)(-1) = 1 + 1 = 2$
- $\|A \text{ part}\| = \sqrt{1^2 + (-1)^2} = \sqrt{2}$ approx 1.414
- $\|B \text{ part}\| = \sqrt{1^2 + (-1)^2} = \sqrt{2}$ approx 1.414
- $\text{sim}(A,B) = 2 / (1.414 \times 1.414) = 2 / 2 = 1.0$

Step 4: adjusted cosine similarity, A vs. C (common items: Movie1 only):

- numerator = $(1)(-2) = -2$
- $\|A \text{ part}\| = \sqrt{1^2} = 1$
- $\|C \text{ part}\| = \sqrt{(-2)^2} = 2$
- $\text{sim}(A,C) = -2 / (1 \times 2) = -1.0$

(A and C rated their one shared movie in opposite directions relative to their own averages, so they're maximally dissimilar — makes sense, this is exactly the "opposite tastes" case the lecture flags as why raw cosine alone is misleading.)

Step 5: predict A's rating on Movie3. Both B and C rated Movie3, so with $k=2$ both are neighbors:

$$\hat{r}_{A,3} = \mu_A + [\text{sim}(A,B) \cdot (r_{B,3} - \mu_B) + \text{sim}(A,C) \cdot (r_{C,3} - \mu_C)] / [|\text{sim}(A,B)| + |\text{sim}(A,C)|]$$

- $r_{B,3} - \mu_B = 4 - 4 = 0$
- $r_{C,3} - \mu_C = 5 - 3 = 2$
- numerator = $(1.0)(0) + (-1.0)(2) = -2$
- denominator = $|1.0| + |-1.0| = 2$
- predicted deviation = $-2/2 = -1$
- predicted rating = $\mu_A + (-1) = 3 - 1 = 2$

So even though B rated Movie3 a 4, C's negative similarity to A pulls the prediction down — C's rating (relative to C's own average) counts *against* Movie3 for A, since A and C tend to disagree. Try re-deriving this with a third neighbor added, or with $k=1$ (only the most similar neighbor), to see how the prediction shifts.

Latent Factor-based Collaborative Filtering

Core idea. Collaborative filtering is really matrix completion. Instead of predicting each missing cell directly from neighbors (as NB-CF does), factorize the n -user $\times$ k -item utility matrix M into two smaller matrices:

$$M \approx F_{\text{user}} \cdot F_{\text{item}}^T$$

where F_{user} is $n \times d$ and F_{item} is $k \times d$, and d is the number of **latent factors** (a hyperparameter, much smaller than n or k). Each row of F_{user} is a user's coordinates in a d -dimensional "taste space"; each row of F_{item} is an item's coordinates in the same space. A predicted rating is just the dot product of the corresponding user and item vectors. Storing F_{user} and F_{item} ($(n+k)d$ numbers) is far cheaper than storing/recomputing a dense completed matrix ($n \times k$ numbers).

Alternating Least Squares (ALS)

```
initialize U (=F_user), V (=F_item)
repeat:
  1. fix U, solve for every row of V independently via least squares (using
  only the ratings that exist for that item)
  2. fix V, solve for every row of U independently via least squares (using
  only the ratings that exist for that user)
until RMSE < tolerance
```

Each row's least-squares problem is independent of the others within a step, so ALS is **parallelizable** — that's its main selling point (good fit for Spark/distributed settings). It's more stable than gradient-descent approaches but each full step is more expensive.

Coordinate (Gradient) Descent (CGD / CD) Instead of solving whole rows at once, CGD updates **one scalar entry** of U or V at a time, holding everything else fixed, to minimize squared error on the known ratings. For entry v_{ij} (row i of V , factor j):

$$v_{ij} = \left[\sum_s u_{sj} \cdot (r_{si} - \sum_{t \neq j} u_{st} v_{it}) \right] / \left[\sum_s u_{sj}^2 \right], \text{ summed over users } s \text{ who rated item } i$$

Derivation of v_{ij} , step by step (a classic "show that..." exam question — reproduce it yourself):

1. Predicted rating: $p_{si} = \sum_t u_{st} v_{it}$ (dot product of user s 's row of U with item i 's row of V).
2. Sum of squared errors over only the ratings that exist for item i :

$$\text{SSE} = \sum_s (r_{si} - \sum_t u_{st} v_{it})^2, \text{ summed over users } s \text{ with a known rating of item } i$$

3. Pull out the one term that involves v_{ij} (factor j of item i 's row), leaving every other term as a constant with respect to v_{ij} :

$$\text{SSE} = \sum_s (r_{si} - u_{sj} v_{ij} - \sum_{t \neq j} u_{st} v_{it})^2$$

4. Differentiate SSE with respect to v_{ij} (chain rule on the squared term):

$$\partial \text{SSE} / \partial v_{ij} = \sum_s -2 \cdot u_{sj} \cdot (r_{si} - u_{sj} v_{ij} - \sum_{t \neq j} u_{st} v_{it})$$

5. Set the derivative to zero (SSE is a convex quadratic in v_{ij} , so this finds its minimum):

$$\sum_s u_{sj} \cdot (r_{si} - u_{sj} v_{ij} - \sum_{t \neq j} u_{st} v_{it}) = 0$$

6. Distribute and collect every term containing v_{ij} on one side:

$$\sum_s u_{sj} r_{si} - v_{ij} \sum_s u_{sj}^2 - \sum_s u_{sj} \sum_{t \neq j} u_{st} v_{it} = 0$$

7. Solve for v_{ij} :

$$v_{ij} = [\sum_s u_{sj} \cdot (r_{si} - \sum_{t \neq j} u_{st} v_{it})] / [\sum_s u_{sj}^2]$$

which is exactly the boxed formula above. Swap the roles of u and v (and s and i) to get the matching update rule for u_{sj} — try writing that one out yourself as a check.

```
initialize U and V (e.g., all ones, for reproducibility)
repeat:
  for every entry in U: update it via the closed-form formula above
  for every entry in V: update it via the closed-form formula above
until the fractional improvement in SSE is below tolerance
```

CGD trades ALS's stability for more granular, cheaper updates — a middle ground between the batch-y ALS and pure stochastic gradient descent (SGD), which updates on one *rating* at a time rather than one *matrix entry*.

ALS vs. CGD (a likely MCQ contrast):

	ALS	CGD
Update unit	a whole row (all d factors)	one scalar entry
Solve method	closed-form least squares	closed-form single-variable minimization
Parallelizable	yes, across rows	yes, but less naturally

	ALS	CGD
Stability	more stable	less stable, but cheaper per step

Surprise library. A scikit-style Python library for recommender systems. `KNNWithMeans` reproduces mean-centered neighborhood-based CF (`sim_options={'name': 'pearson', 'user_based': True/False}` switches between user- and item-based). Also has matrix-factorization models (e.g., SVD). Useful for sanity-checking your own from-scratch implementation against a trusted library — expect small numeric differences from tie-breaking and implementation details, not identical numbers.

Worked by hand: one coordinate-descent update

Toy 2-user x 2-item matrix M (both entries known), $d=1$ latent factor, both U and V initialized to all ones (matching the exercise's convention):

	Item1	Item2
User1	4	-
User2	-	3

$U = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ (one factor per user), $V = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ (one factor per item).

Update v_{11} (row 1 of V , i.e. Item1's factor, the only factor since $d=1$). Only User1 rated Item1, so s ranges over $\{\text{User1}\}$ only:

$$v_{11} = [\sum_s u_{s1} \cdot (r_{s1} - \sum_{t \neq 1} u_{st} v_{1t})] / [\sum_s u_{s1}^2]$$

With $d=1$ there is no $t \neq 1$ term (it's an empty sum = 0), so this simplifies to:

- numerator = $u_{11} \times (r_{11} - 0) = 1 \times 4 = 4$
- denominator = $u_{11}^2 = 1^2 = 1$
- $v_{11} = 4 / 1 = 4$

Update v_{21} (Item2's factor). Only User2 rated Item2:

- numerator = $u_{21} \times r_{22} = 1 \times 3 = 3$
- denominator = $u_{21}^2 = 1$
- $v_{21} = 3$

V is now $\begin{bmatrix} 4 \\ 3 \end{bmatrix}$. Next you'd sweep U the same way (update u_{11} using the new v_{11} , then u_{21} using the new v_{21}), recompute SSE, and check whether it dropped enough to stop.

Sanity check against ALS/CD intuition: with only one rating touching each item and each user in this tiny example, one coordinate sweep is enough to drive the reconstruction error to zero ($U \times V^T$ will now exactly reproduce M at the observed cells). In a real, denser matrix, each entry is pulled toward *many* ratings at once, so it takes several full sweeps over all of U and V before SSE stops improving — try extending this toy example to 3 users x 3 items with more overlap and see how many sweeps it takes to converge.

Self-check questions

Cover the answer column and answer from memory first.

#	Question	Answer
1	Which V of Big Data refers to data trustworthiness/quality?	Veracity
2	Why does increasing item count hurt FIM runtime more than increasing transaction count?	Runtime is $O(N \times 2^d)$ — exponential in items (d), linear in transactions (N)
3	What property lets Apriori skip candidates without counting them?	Downward closure: every subset of a frequent itemset must itself be frequent, so a candidate with any infrequent subset can be pruned
4	Why is FP-Growth generally faster than Apriori in practice?	It grows itemsets only from projected (actually existing) sub-databases instead of generating and testing candidates that may not appear in the data
5	Rule $A \rightarrow B$ has support 20, $\text{sup}(A) = 40$, and $\text{relSup}(B) = 0.1$ over $N = 200$ transactions. Find confidence and lift.	confidence = $20/40 = 0.5$; lift = $0.5 / 0.1 = 5$

#	Question	Answer
6	Why mean-center before computing similarity in NB-CF?	Raw ratings confound taste with a user's personal rating scale (e.g., someone who rates everything a 5); subtracting each user's mean isolates relative preference
7	In user-based CF, what do you weight the neighbors' (mean-centered) ratings by?	Their similarity to the target user
8	Why is item-based CF often more stable than user-based CF?	Items tend to cluster by genre/category, giving more consistent item-item similarity, whereas a user can like several genres at once, diluting user-user similarity
9	What does NDCG add on top of DCG?	Normalization by the ideal DCG (IDCG), making scores comparable across users on a 0-1 scale
10	In latent factor CF, what does the dimension d control?	The number of latent factors — the size of the compressed "taste space" each user/item vector lives in
11	What is the key advantage of ALS over pure gradient-based methods?	Each row update is an independent closed-form least-squares solve, so it parallelizes cleanly across users/items
12	What does coordinate descent update at each step, and how does that differ from ALS?	One scalar matrix entry at a time (vs. ALS updating a whole row via least squares)
13	What's the difference between online and offline recommender evaluation?	Online: real users interact live (e.g., A/B test), expensive but realistic; offline: reuses already-collected ratings, cheap but can be stale
14	Why does Spark outperform plain MapReduce on iterative algorithms like ALS?	Spark keeps intermediate data in memory across steps instead of writing to disk after every map/reduce stage

#	Question	Answer
15	What does "data locality" mean in Hadoop, and why does it matter?	Scheduling computation on/near the node that already holds the data, since moving data across the network is far more expensive than moving a small computation

Hands-on exam strategy

Since the prof said the hands-on half is based on the exercises, the likely exam pattern is the same as your Assignment 1 on the Sushi dataset: load a new dataset, run FIM/ARM on it, run one or both CF methods on a utility matrix, and interpret the results for a business question.

Functions you should be able to write from memory (structure, not necessarily every edge case):

- FIM: `to_sparse` (transactions -> sparse matrix), `brute` / `apriori` / `fpgrowth` (frequent itemsets given minsup), `assoc` (rules given itemsets + minconf)
- NB-CF: `raw_cos` and adjusted-cosine similarity, `user_complete` / `item_complete` (fill in a utility matrix), `recommend` (top-N unrated items), `dcg` / `ndcg`
- LF-CF: `als`, `cd` (coordinate descent), `recommend_als` / `recommend_cd`

Likely gotchas from your own exercises worth re-checking:

- Sorting conventions matter for the autograder-style checks: frequent itemsets sorted by decreasing support, then decreasing length, then alphabetically; recommendations sorted by decreasing predicted rating, then by item id as a tiebreak.
- Only use *commonly rated* items when computing similarity — never impute missing ratings with 0 or a mean before computing cosine/adjusted cosine.
- Watch absolute vs. relative support/confidence, especially when mixing your own code with pyFIM (`supp` sign convention) or Surprise.
- `assoc` 's lift formula needs the *consequent*'s relative support as the denominator, not the antecedent's.

Practice plan: work through the companion review notebook (sent separately, and saved into your BDCC 3301 folder) — it has the same function signatures as your solved exercises but with new toy data and asserts, so you're rebuilding each one from the

algorithm, not recalling your old code.